MANTENIMIENTO, RESPALDO Y RECUPERACION

TP UNIDAD 6

Joaquin Antonio Mendez Reynos

Materia

Bases de Datos II - Tecnicatura Universitaria en Programación

Motor

PostgreSQL 16+

1) Objetivo

Integrar respaldo, monitoreo, mantenimiento y recuperación sobre la base Food Store. Se construye un procedimiento para generar una copia, registrar métricas, restaurar en un entorno descartable (foodstore_restore_test) y medir el tiempo de recuperación.

2) Especificación y plan de mantenimiento (SPEC_U6)

Tarea mínima	Frecuencia propuesta	Evidencia	Condición para considerar terminada
Respaldo de foodstore	Diaria	Archivo .dump con fecha/hora + log	pg_dump finaliza OK, pg_restore --list válido, SHA256 registrado
Chequeo de salud	Diaria	health_check.sql	Tamaño de base, actividad de conexiones, estado de tablas, consultas activas sin errores
Revisión de mantenimiento	Semanal	maintenance.sql	Revisión previa obligatoria, acción segura justificada, sin VACUUM FULL
Restauración de prueba	Semanal	foodstore_restore_test validada	Conteos coincidentes, productos_invalidos=0, detalles_invalidos=0, RPO/RTO registrados

3) Respaldo automatizable

Se utiliza el comando base en formato custom (-Fc) para poder restaurar con pg_restore:

```
pg_dump -h localhost -p 5432 -U postgres -d foodstore -Fc -f backups/foodstore_FECHA_HORA.dump
```

Se entregan los scripts:
- backup_foodstore.ps1 (Windows)
- backup_foodstore.sh (Linux)

Características:
- Sin contraseñas en los scripts. Uso de pgpass.conf (Windows) o ~/.pgpass con permisos 600 (Linux)
- Se genera archivo .parcial y se renombra al finalizar correctamente
- Se verifica con pg_restore --list
- Se genera huella SHA256
- Se registra el resultado en carpeta logs
- Se detiene con error si pg_dump/pg_restore fallan

4) Monitoreo y mantenimiento health_check.sql

Controles mínimos:

- Tamaño de la base: pg_database_size('foodstore')
- Actividad de conexiones: pg_stat_database
- Estado de tablas: pg_stat_user_tables (n_live_tup, n_dead_tup, last_autovacuum, last_autoanalyze)
- Consultas activas: pg_stat_activity (distintas a idle)

maintenance.sql

- Revisión previa obligatoria sobre tuplas vivas/muertas y fechas de autovacuum/analyze
- Acción de mantenimiento segura y justificada (ej. VACUUM (ANALYZE)) solo si la observación lo requiere
- Regla: VACUUM FULL NO se utiliza como mantenimiento periódico

5) Simulacro de desastre contra reloj

Restauración en entorno descartable:

```
createdb -h localhost -p 5432 -U postgres foodstore_restore_test
pg_restore -h localhost -p 5432 -U postgres -d foodstore_restore_test --no-owner --no-privileges backups/ULTIMO_RESPALDO.dump
```

Validaciones mínimas sobre foodstore_restore_test:

```
SELECT 'categoria' AS tabla, COUNT(*) AS filas FROM categoria
UNION ALL SELECT 'cliente', COUNT(*) FROM cliente
UNION ALL SELECT 'producto', COUNT(*) FROM producto
UNION ALL SELECT 'pedido', COUNT(*) FROM pedido
UNION ALL SELECT 'detalle_pedido', COUNT(*) FROM detalle_pedido;

SELECT COUNT(*) AS productos_invalidos FROM producto WHERE precio < 0 OR stock < 0;
SELECT COUNT(*) AS detalles_invalidos FROM detalle_pedido WHERE cantidad <= 0 OR precio_unitario < 0;
```

Resultado:

- Conteos por tabla validados contra pre-respaldo: Coincidentes
- productos_invalidos: 0
- detalles_invalidos: 0
- RPO observado: 24 horas (última copia diaria)
- RTO observado: 12 minutos (restauración + validación)

6) Problema real encontrado

No se encontraron problemas durante el procedimiento. Se verificó la correcta ejecución de pg_dump, pg_restore y las validaciones mínimas sin errores.

7) Mejora concreta propuesta

Implementar verificación automática del backup mediante pg_restore --list y registro en bitácora, programar el respaldo diario con retención de 14 días, ajustar umbrales de autovacuum por tabla según carga real y mantener siempre la restauración periódica en entorno descartable para validar la recuperabilidad.

8) Uso de OpenCode

Parte asistida con OpenCode: generación de la estructura base de los scripts (backup_foodstore, health_check, maintenance, restore_test) siguiendo SPEC_U6.md. Verificación antes de aceptar: lectura línea por línea de cada script, corroboración de que no contienen contraseñas, validación de comandos contra PostgreSQL 16, pruebas en entorno descartable foodstore_restore_test y subordinación del uso de la herramienta a revisión y prueba humana.

Repositorio Git

https://github.com/VIKROOM/informe_TPU6_Mendez_Reynoso.git